

به نام خدا

گزارش سوم آزمایشگاه
سیستم عامل

سید علیرضا میرشفیعی - ۸۱۰۱۰۱۵۳۲

محمد تقی زاده - ۸۱۰۱۰۲۵۸۹

محمد صدرا عباسی - ۸۱۰۱۰۱۴۶۹

ساختار PCB در XV6 و در فایل proc.h :

```
// Per-process state
struct proc {
    uint sz;                // Size of process memory (bytes)
    pde_t* pgdir;           // Page table
    char *kstack;           // Bottom of kernel stack for this process
    enum procstate state;   // Process state
    int pid;                // Process ID
    struct proc *parent;    // Parent process
    struct trapframe *tf;   // Trap frame for current syscall
    struct context *context; // switch() here to run process
    void *chan;             // If non-zero, sleeping on chan
    int killed;             // If non-zero, have been killed
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;      // Current directory
    char name[16];          // Process name (debugging)

    int priority;           // [NEW] Process priority
};
```

(فیلد priority برای هندل کردن یکی از سیستم کال ها در فاز قبلی اضافه شده)
ساختار نشان داده شده در کتاب:

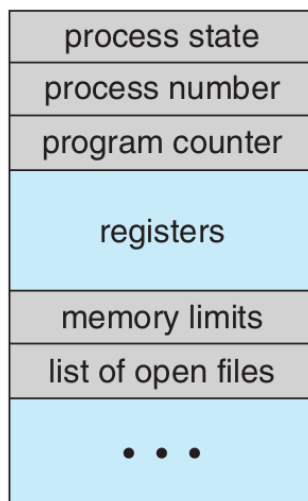


Figure 3.3 Process control block (PCB).

داده ها و فیلد و های مشابه:

- `enum procstate state` : Process state در هر دو ساختار وجود دارند که به وضعیت پردازش اشاره دارند
- `int pid` : Process number که شناسه پردازش میباشد.
- `struct trapframe *tf->eip` : Program counter درون استراکت `trapframe` فیلد `eip` همان `program counter` هست که آدرس دستور بعدی می باشد.
- `char *kstack` , `pde_t* pgdir` , `uint sz` : Registers که برابر اندازه حافظه ، `pgdir` همان `page directory` و `kstack` آدرس ابتدای `kernel stack` مربوط به پردازش است.
- `struct file *ofile` : List of open files آرایه ای از اشاره گر ها به فایل هایست که پردازش باز کرده.

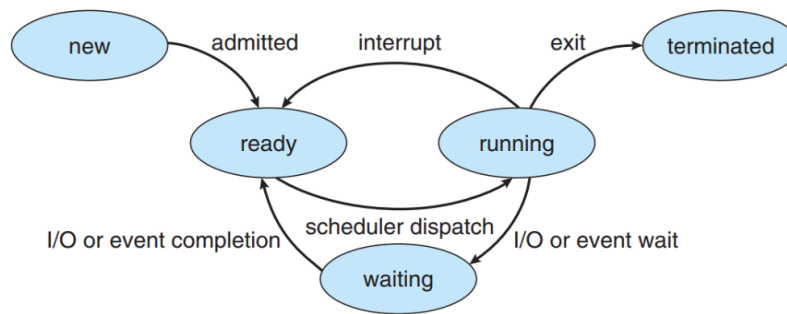
وضعیت های پردازش:

```
enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
```

- `UNUSED` : پردازش مورد استفاده قرار نگرفته
- `EMBRYO` : پردازش به تازگی در حال ایجاد شدن هست
- `SLEEPING` : پردازش منتظر یک `event` است
- `RUNNABLE` : آماده اجرا اما هنوز اجرا نشده
- `RUNNING` : در حال اجرا روی پردازنده
- `ZOMBIE` : پردازش پایان یافته اما منابعش هنوز آزاد نشده

(۲)

```
enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
```



شکل ۱. چرخه وضعیت یک پردازشگر.

- UNUSED : (معادل ندارد)
- EMBRYO : (new)
- SLEEPING : (waiting)
- RUNNABLE : (ready)
- RUNNING : (running)
- ZOMBIE : (terminated)

(۳)

علاوه بر `userinit` که تابع اولین پردازش هست و پس از تخصیص فضا و تنظیمات اولیه وضعیت پردازش را به `runnable` یا همون `ready` تغییر می دهد، تابع `fork` نیز این `transition` را انجام می دهد:

userinit():

```

void
userinit(void)
{
    struct proc *p;
    extern char _binary_initcode_start[], _binary_initcode_size[];

    p = allocproc();
    initproc = p;
    //some codes
    acquire(&ptable.lock);

```

```

p->state = RUNNABLE;
release(&ptable.lock);
}

```

ابتدا تابع allocproc فراخوانی می شود که در آن با جستجو بر روی لیست پردازش ها یک پردازش که در حالت unused قرار دارد پیدا می شود. و state آن به EMBRYO که معادل new است تغییر می کند. در ادامه با راه اندازی حافظه مجازی و بارگذاری کد کاربر و تنظیمات اولیه trapframe پردازش آماده به اجرا توسط scheduler می شود و حالت آن از new به RUNNABLE یا هما ready تغییر می کند. در نتیجه تغییر حالات پردازش به شکل زیر تغییر می کند:

UNUSED -> EMBRYO -> RUNNABLE

fork():

```

int
fork(void)
{
    int i, pid;
    struct proc *np;
    struct proc *curproc = myproc();

    if((np = allocproc()) == 0){
        return -1;
    }

    //some codes

    pid = np->pid;
    acquire(&ptable.lock);
    np->state = RUNNABLE;
    release(&ptable.lock);

    return pid;
}

```

این تابع نیز ابتدا allocproc را فراخوانی می کند که باعث allocate کردن یه پردازش جدید با تخصیص حافظه و stack و تغییر آن از حالت UNUSED به EMBRYO می شود در ادامه با کپی حافظه والد و متا دیتا های دیگر و فایل های باز والد، حالت پردازش از EMBRYO به RUNNABLE تغییر می کند بنابر این مشابه تابع قبل تغییر حالات به صورت:

UNUSED -> EMBRYO -> RUNNABLE

خواهد بود.

(۴)

```
struct {  
    struct spinlock lock;  
    struct proc proc[NPROC];  
} ptable;
```

مقدار NPROC که به صورت دیفاین هست حداکثر تعداد پردازش ها برای ptable را تعیین می کند که برای آن در param.h مقدار ۶۴ تعیین شده :

```
define NPROC    64 // maximum number of processes#
```

زمانی که تعداد پردازش ها از مقدار NPROC عبور کند، در proc.c زمانی که تابع allocproc برای fork یک پردازش جدید فراخوانده میشود، سطح کرنل صرفاً این اجازه داده نمیشود و allocproc شکست میخورد. در نتیجه این شکست در سمت کاربر بسته به کد، مقدار 1- برگردانده میشود، که همان نشانه شکست fork میباشد.

(۵)

از آنجایی که ptable یک داده‌ی shared بین پردازنده‌هاست، بنابراین دسترسی به آن باید به صورت حفاظت‌شده و با استفاده از lock انجام شود؛ در غیر این صورت می‌تواند منجر به race condition و تداخل‌های شدید داده‌ای شود (مثلاً اگر چند پردازنده همزمان یک پردازشی مشترک را برای اجرا انتخاب کنند).

```

void

acquire(struct spinlock *lk)

{

    pushcli(); // disable interrupts to avoid deadlock.

    if(holding(lk))

        panic("acquire");

    while(xchg(&lk->locked, 1) != 0)

        ;

    __sync_synchronize();

    lk->cpu = mycpu();

    getcallerpcs(&lk, lk->pcs);

}

```

حتی اگر تنها یک پردازنده داشته باشیم، باز هم دسترسی به ptable باید به صورت اتومیک صورت گیرد و در حین آن وقفه‌ها غیرفعال شوند وگرنه تداخل وقفه‌ها می‌تواند رخ دهد که منجر به تغییرات ناخواسته در وضعیت پردازنده می‌شود. مثلاً اگر کرنل در حال تغییر وضعیت یک پردازنده به SLEEPING است ولی هنوز عملیات سوییچ انجام نشده است. اگر در همین لحظه یک وقفه (مثل پایان کار دیسک) رخ دهد و تابع wakeup برای بیدار کردن همان پردازنده اجرا شود، به دلیل اینکه وقفه‌ها بسته نبوده‌اند، ممکن است این wakeup گم شود چون وضعیت پردازنده در آن لحظه هنوز به صورت RUNNING است و اگر بعد از هندل شدن وقفه به اجرای کد خود ادامه دهد به وضعیت SLEEP می‌رود و می‌تواند همیشه در آن حالت باقی بماند. بنابراین، همچنان استفاده از قفل به منظور غیرفعال کردن وقفه‌ها ضروری است.

(۶)

این موضوع کاملاً بستگی به مکان آن پردازش در ptable.proc و همچنین مکان ایندکس scheduler میباشد
اگر مکان پردازش در ptable.proc بعد از مکان ایندکس scheduler باشد در همان iteration پردازش اجرا خواهد شد.

اما اگر مکان پردازش در ptable.proc قبل از مکان ایندکس scheduler باشد، در iteration بعدی scheduler از این پردازش گذر میکند تا آن را اجرا کند. در هر دو حالت تک هسته ای و چند هسته ای این جواب صادق است به دلیل استفاده از صف اشتراکی و قفل گذاری آن در اسکچولر باید پردازنده فعلی تا انتهای صف اشتراکی خود پیمایش کند و بعد قفل را آزاد می کند پس عملاً مانند حالت تک هسته عمل می کند.

(۷)

```
struct context {  
    uint edi;  
    uint esi;  
    uint ebx;  
    uint ebp;  
    uint eip;  
};
```

- edi - Extended Destination Index به عنوان مقصد استفاده می شود
- esi - Extended Source Index به عنوان مبدا داده استفاده می شود
- ebx - Base Register یک رجیستر چند منظوره که در نگهداری داده ها، اشاره گرها و... ممکنه استفاده بشه
- ebp - Base Pointer نشان دهنده ابتدای stack frame
- eip - Extended Instruction Pointer همان program counter میباشد.

(۸)

نام این رجیستر ساختار در context همان eip است. با فراخوانی swtch در scheduler کد اسمبلی زیر فراخوانی می شود که در ادامه به بررسی آن می پردازیم

```
.globl swtch
swtch:
    movl 4(%esp), %eax
    movl 8(%esp), %edx

    # Save old callee-saved registers
    pushl %ebp
    pushl %ebx
    pushl %esi
    pushl %edi

    # Switch stacks
    movl %esp, (%eax)
    movl %edx, %esp

    # Load new callee-saved registers
    popl %edi
    popl %esi
    popl %ebx
    popl %ebp
    ret
```

ابتدا آرگومان ها روی استک پوش می شوند که در اولی آدرس **old context قرار دارد و در دومی context *new. سپس ۴ رجیستر که نشان دهنده وضعیت پردازش قبلی است بر روی استک آن قرار می گیرند. سپس به این دو دستور مهم می رسیم:

movl %esp, (%eax): مقدار فعلی اشاره گر پشته (esp) را در آدرسی که eax به آن اشاره می‌کند، ذخیره می‌کند. eax حاوی old است. پس عملاً داریم می‌گوییم: $old = esp$; با این کار، آدرس بالای پشته‌ی پردازشی قدیمی (که الان شامل تمام رجیسترهای ذخیره شده است) را در متغیر context آن پردازشی ذخیره می‌کنیم تا بعداً بتوانیم به آن برگردیم.

movl %edx, %esp: مقدار esp را تغییر می‌دهیم و برابر با edx قرار می‌دهیم. edx حاوی آدرس context پردازشی جدید است.

سپس مقادیر مربوط به استک جدید به ترتیب در رجیسترهای پردازنده قرار می‌گیرند و دستور ret در انتهای تابع switch، مقداری را از بالای استک فعلی (که اکنون استک پردازشی جدید است) برداشته و آن را درون رجیستر شمارنده برنامه یا همان eip قرار می‌دهد. این مقدار در واقع همان آدرس بازگشت است که قبلاً هنگام توقف پردازشی ذخیره شده بود، بنابراین با این کار پردازنده به آن آدرس پرش کرده و اجرای پردازشی جدید را دقیقاً از همان نقطه‌ای که متوقف شده بود (خط بعد از فراخوانی switch) از سر می‌گیرد.

(۹)

اگر وقفه‌ها در حلقه مربوط به scheduler غیرفعال باشند، سیستم دچار deadlock می‌شود. به طور مثال اگر همه پردازش‌ها تابع sleep را فراخوانی کرده و به حالت SLEEPING رفته باشند (مثلاً منتظر ورودی دیسک یا کیبورد باشند)، تنها راه بیدار شدن آن‌ها وقوع یک وقفه سخت‌افزاری است که تابع wakeup را اجرا کند. حال اگر وقفه‌ها غیرفعال باشند، پردازنده اصلاً متوجه این سیگنال‌ها نمی‌شود تا وضعیت پردازش‌ها را به RUNNABLE تغییر دهد؛ در نتیجه scheduler مدام در حلقه اجرا می‌شود و هیچ پردازشی آماده‌ای برای اجرا پیدا نمی‌کند و سیستم فریز می‌شود.

تابع sti با تنظیم فلگ وقفه (IF) در رجیستر CPU باعث می‌شود که پردازنده به وقفه‌های سخت‌افزاری واکنش نشان دهد. در مقابل آن تابع cli وجود دارد که وقفه‌ها را غیرفعال می‌کند؛ این عمل هنگام گرفتن قفل ptable برای دسترسی به ناحیه مشترک انجام می‌شود تا فرایند انتخاب پردازشی به صورت اتومیک صورت گیرد. بنابراین، اگر تابع sti در ابتدای حلقه scheduler فراخوانی نشود (تا وقفه‌هایی را که قبلاً بسته شده بودند باز کند)، وقفه‌های سخت‌افزاری به طور کامل نادیده گرفته می‌شوند.

(۱۰)

```
// from lapic[TICR] and then issues an interrupt.  
// If xv6 cared more about precise timekeeping,  
// TICR would be calibrated using an external time source.  
lapicw(TDCR, X1);
```

```
lapicw(TIMER, PERIODIC | (T_IRQ0 + IRQ_TIMER));  
lapicw(TICR, 10000000);
```

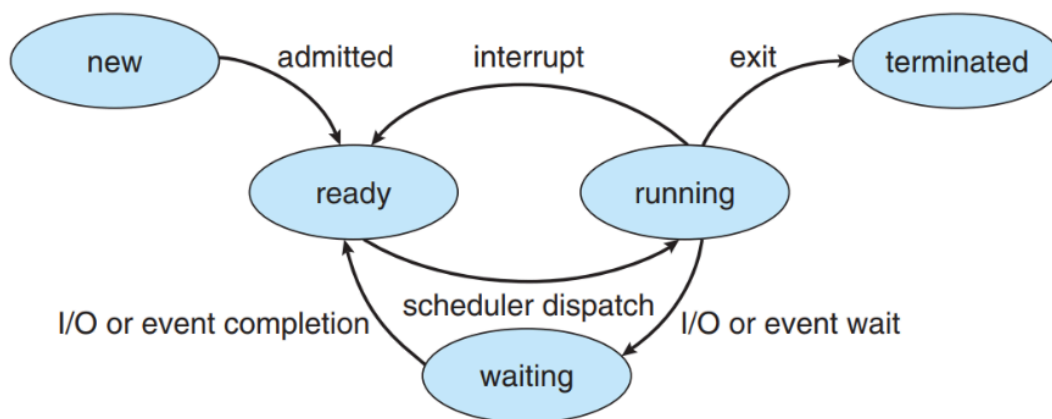
در این کد می توان مقدار اولیه تایمر که برابر 10,000,000 است و همچنین مقدار تقسیم کننده آن که برابر یک کلاک هست را مشاهده کرد.

در نتیجه بسته بر اینکه کلاک با چه فرکانسی باشد این مقدار می تواند متغیر باشد.

در صورتی که فرض کنیم مقدار کلاک برابر 1GHz (شبه ساز qemu همین مقدار را تعیین می کند) باشد، تایمر 10 میلی ثانیه خواهد بود یعنی در هر 10 میلی ثانیه تایمر یک وقفه می دهد.

لینک سوالات : <https://chatgpt.com/share/692dd607-3470-8008-91e0-ea5331c52f9a>

(۱۱)



تغییر وضعیت پردازش از حالت running به ready: با هر بار وقفه تایمر، تابع yield فراخوانی می شود. این تابع ابتدا وضعیت پردازش فعلی را از RUNNING به RUNNABLE (معادل Ready) تغییر می دهد و سپس با فراخوانی sched، عمل Context Switch به اسکچولر را انجام می دهد تا پردازنده در اختیار پردازش دیگری قرار گیرد.

(۱۲)

در سوال ۱۰ مشاهده کردیم که فرکانس تایمر ۱۰۰ هرتز تنظیم شده است، به این معنی که در هر ثانیه ۱۰۰ وقفه تایمر رخ می‌دهد (هر ۱۰ میلی‌ثانیه یک وقفه).

در پیاده‌سازی xv6، با وقوع هر وقفه تایمر، اگر پردازش‌ای در حال اجرا باشد، وقفه صادر شده و تابع yield() فراخوانی می‌شود تا پردازنده را رها کند. از آنجا که در زمان‌بند پیش‌فرض xv6 هیچ شمارنده‌ای برای نگه‌داشتن پردازش برای چندین تیک متوالی وجود ندارد، پردازش با اولین وقفه تایمر تعویض می‌شود. بنابراین کوانتوم زمانی در این سیستم عامل دقیقاً برابر با یک تیک یا ۱۰ میلی‌ثانیه است.

(۱۳)

تابع wait برای جلوگیری از هدررفت منابع پردازشی و ممانعت از Busy Waiting، زمانی که تشخیص می‌دهد پردازش فرزند وجود دارد اما هنوز هیچ‌کدام خاتمه نیافته‌اند، از تابع sleep استفاده می‌کند.

این تابع وضعیت پردازش والد را به SLEEPING تغییر می‌دهد تا پردازنده از آن گرفته شده و به پردازش‌های دیگر داده شود، تا زمانی که یکی از فرزندان exit کند.

(۱۴)

علاوه بر wait، تابع sleep کاربرد اصلی در همگام‌سازی (Synchronization) و مدیریت دسترسی به منابع مشترک دارد.

یک نمونه بارز دیگر، استفاده در مکانیزم Pipe است؛ زمانی که بافر پایپ خالی است و پردازش‌ای قصد خواندن از آن را دارد، تابع sleep فراخوانی می‌شود تا خواننده به خواب برود و منتظر بماند تا نویسنده (Writer) داده‌ای در پایپ بنویسد.

(۱۵)

الف) تابع wakeup. این تابع در سطح کرنل وظیفه دارد پردازشهایی را که منتظر یک رویداد خاص (روی یک کانال مشخص) هستند، پیدا کرده و بیدار کند.

ب) این تابع وضعیت پردازش را از SLEEPING (معادل Waiting) به RUNNABLE (معادل Ready) تغییر می‌دهد تا در نوبت‌دهی بعدی توسط زمان‌بند انتخاب شود.

ج) بله، تابع kill نیز می‌تواند این گذار را انجام دهد. اگر پردازش خواب باشد و دستور kill دریافت کند، بیدار می‌شود (به وضعیت RUNNABLE می‌رود) تا بتواند سیگنال خاتمه را پردازش کرده و خارج شود.

(۱۶)

سیستم‌عامل xv6 در مواجهه با پردازش‌های یتیم (Orphan) از روش فرزندخواندگی (Reparenting) استفاده می‌کند.

زمانی که یک پردازش exit می‌کند، قبل از خاتمه کامل، تمام فرزندان زنده‌ی خود را به پردازش‌ی init واگذار می‌کند. پردازش‌ی init (اولین پردازش سیستم) در یک حلقه بی‌نهایت دائماً تابع wait را فراخوانی می‌کند تا هر زمان که یکی از این فرزندان یتیم خاتمه یافت، منابع آن‌ها را آزاد و پاک‌سازی (Clean up) کند و از ایجاد پردازش‌های زامبی جلوگیری شود.

(۱۷)

ساختار struct cpu که در فایل proc.h تعریف شده، وضعیت اختصاصی هر هسته پردازنده را نگه می‌دارد. فیلدهای کلیدی آن عبارتند از:

- **apicid**: شناسه سخت‌افزاری منحصر به فرد (Local APIC ID) برای تشخیص هسته.
- **scheduler**: اشاره‌گر به کانتکست (Context) زمان‌بند؛ وقتی هسته می‌خواهد از اجرای پردازش به اجرای کد زمان‌بند سوئیچ کند، رجیسترها در اینجا ذخیره می‌شوند.
- **ts**: ساختار Task State که x86 برای یافتن پشته در زمان وقوع وقفه استفاده می‌کند.
- **gdt**: جدول توصیف‌گر سراسری (Global Descriptor Table) اختصاصی برای هر هسته.
- **started**: نشان‌دهنده روشن شدن و شروع به کار هسته.
- **ncli** و **intena**: متغیرهایی برای مدیریت تودرتوی قفل‌ها و وضعیت فعال/غیرفعال بودن وقفه‌ها.
- **proc**: اشاره‌گر به پردازش‌ای که هم‌اکنون روی این هسته در حال اجراست.

تغییر کوانتوم زمانی نوبت گردشی و نتیجه تست آن

برنامه کاربر که چند پردازش می سازد و هرکدام را مشغول می کند:

```
int main(){
    printf(1, "\n[TEST] Testing ticks...\n");
    for(int i = 0; i < PROCESS_COUNT; i++){
        int pid = fork();
        if(pid < 0){
            printf(1, "Fork failed!\n");
            exit();
        }
        if(pid == 0){
            cpu_bound_task(getpid());
            exit();
        }
    }
    for(int i = 0; i < PROCESS_COUNT; i++)
        wait();
    printf(1, "[PASS] Ticks test completed.\n");
    exit();
}
```

نتیجه اجرا با یک هسته و ۳ پردازنده : نوبت گردشی به درستی و با کوانتوم زمانی ۳ تیک یعنی 30ms به درستی انجام شده

```
[ST] Testing ticks...
[CPU 0] PID 4 (E-Core) | Tick: 1
[CPU 0] PID 4 (E-Core) | Tick: 2
[CPU 0] PID 4 (E-Core) | Tick: 3
[CPU 0] PID 5 (E-Core) | Tick: 1
[CPU 0] PID 5 (E-Core) | Tick: 2
[CPU 0] PID 5 (E-Core) | Tick: 3
[CPU 0] PID 6 (E-Core) | Tick: 1
[CPU 0] PID 6 (E-Core) | Tick: 2
[CPU 0] PID 6 (E-Core) | Tick: 3
[CPU 0] PID 7 (E-Core) | Tick: 1
[CPU 0] PID 7 (E-Core) | Tick: 2
[CPU 0] PID 7 (E-Core) | Tick: 3
[CPU 0] PID 4 (E-Core) | Tick: 1
[CPU 0] PID 4 (E-Core) | Tick: 2
[CPU 0] PID 4 (E-Core) | Tick: 3
[CPU 0] PID 5 (E-Core) | Tick: 1
[CPU 0] PID 5 (E-Core) | Tick: 2
[CPU 0] PID 5 (E-Core) | Tick: 3
[CPU 0] PID 6 (E-Core) | Tick: 1
[CPU 0] PID 6 (E-Core) | Tick: 2
[CPU 0] PID 6 (E-Core) | Tick: 3
[CPU 0] PID 7 (E-Core) | Tick: 1
[CPU 0] PID 7 (E-Core) | Tick: 2
[CPU 0] PID 7 (E-Core) | Tick: 3
[CPU 0] PID 4 (E-Core) | Tick: 1
[CPU 0] PID 4 (E-Core) | Tick: 2
[CPU 0] PID 4 (E-Core) | Tick: 3
[CPU 0] PID 5 (E-Core) | Tick: 1
[CPU 0] PID 5 (E-Core) | Tick: 2
[CPU 0] PID 5 (E-Core) | Tick: 3
[CPU 0] PID 6 (E-Core) | Tick: 1
[CPU 0] PID 6 (E-Core) | Tick: 2
[CPU 0] PID 6 (E-Core) | Tick: 3
[CPU 0] PID 7 (E-Core) | Tick: 1
[CPU 0] PID 7 (E-Core) | Tick: 2
[CPU 0] PID 7 (E-Core) | Tick: 3
[CPU 0] PID 4 (E-Core) | Tick: 1
[CPU 0] PID 4 (E-Core) | Tick: 2
[CPU 0] PID 4 (E-Core) | Tick: 3
[CPU 0] PID 5 (E-Core) | Tick: 1
[CPU 0] PID 5 (E-Core) | Tick: 2
[CPU 0] PID 5 (E-Core) | Tick: 3
[CPU 0] PID 6 (E-Core) | Tick: 1
[CPU 0] PID 6 (E-Core) | Tick: 2
[CPU 0] PID 6 (E-Core) | Tick: 3
[CPU 0] PID 7 (E-Core) | Tick: 1
[CPU 0] PID 7 (E-Core) | Tick: 2
[CPU 0] PID 7 (E-Core) | Tick: 3
[PASS] Ticks test completed.
```

نتیجه اجرا با دو هسته زوج و ۲ پردازشگر: بدیهی است که در این حالت بدون قفل گذاری صحیح و صرفا چاپ آنها در ترپ شکل خروجی نامرتب می شود اما همچنان خروجی نشان می دهد که زمانبندی و توزیع پردازش ها به شکل کاملا صحیح انجام شده

[illegible]